EXAMEN

Semestre : 1 [X] 2

Session : Principale

Module : **Architecture des SI II (Spring)**
Enseignant(s) : **UP ASI (Spring Team)**
Classe(s): **4ème TWIN.**

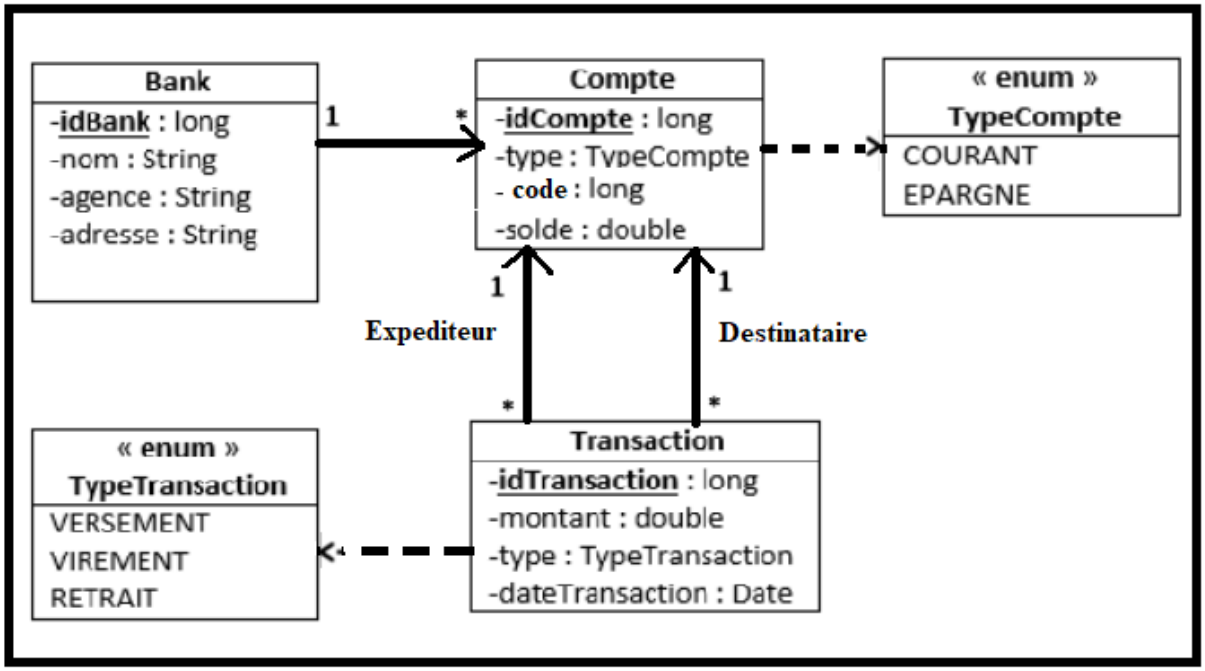
Documents autorisés : **OUI** [X] NON [ ] Nombre de pages : **3**

Calculatrice autorisée : OUI [ ] **NON** [X] Internet autorisée : OUI [ ] **NON** [X]

Date : 09/01/2023 **Heure 9h** **Durée : 1h30**

La validation de l'épreuve est appliquée sur la base d'un code source exécutable. Aucun code source non fonctionnel n'est comptabilisé lors de la validation.

On se propose de mettre en place une application de gestion des transactions des comptes bancaire et des Banques. Ci-dessous, le diagramme de classes.



I. (6 points)

1) Implémenter les entités qui permettent de générer le schéma de la base de données comme illustré dans le diagramme de classes sachant que :
   - Les **identifiants** sont auto-générés avec une séquence pour toutes les tables.
   - L'**énumération** doit être stockée en tant que chaîne de caractères dans la base.

II. (14 points)

Pour chacune des questions suivantes, développer le code nécessaire dans une classe annotée par **@RestController** qui fait appel aux différents services.

N.B : Chaque question doit être testée sur Postman ou Swagger.

Le projet doit respecter l'architecture en couches vu au niveau du cours.

1) Ajouter **2 Banks** ayant les détails ci-dessous en respectant la signature suivante **(/1)** :

```
public Bank ajouterBank(Bank bank);
```

nom	agence	adresse
BNA	Beja	Centre Ville
ATB	Ariana	Borej Baccouche

2) Ajouter et affecter **4 comptes** ayant les détails ci-dessous en respectant la signature suivante **(/1.5)** :

```
public Compte ajouterCompteEtAffecterAAgence(Compte compte,
                                             String agenceBank);
```

type	code	solde (DT)	agence
COURANT	1	5.000	Beja
EPARGNE	2	10.500,752	Beja
COURANT	3	150.000	Beja
COURANT	4	300	Ariana

3) Ajouter **4 Transactions** ayant les détails ci-dessous en respectant les signatures suivantes **(/5)** :

montant (DT)	type	expéditeur (code Compte)	destinataire (code Compte)
4.000	RETRAIT	1	-
30.000	VERSEMENT	-	2
10.000	VIREMENT	3	2
20.500	VIREMENT	3	4

```
public String ajouterVirement(Transaction transaction); (/2)
```

- Si le type de transaction est un **VIREMENT** et l'Expéditeur est un compte **EPARGNE** la méthode retourne : « On ne peut pas faire un virement à partir d'un compte épargne »
- Si le type de transaction est un **VIREMENT** et l'Expéditeur n'est pas un compte **EPARGNE** mais le solde de compte est inférieur au montant transféré + 3DT la méthode retourne : « On ne peut pas faire un virement : Solde

insuffisant»

- Si les conditions d'ajout sont toutes vraies la méthode retourne un message respectant cette forme :
  « VIREMENT de 10.000 DT de compte 3 vers le compte 2 approuvé avec succès. »

`public String ajouterRetrait(Transaction transaction);` (/1.5)

- Si le type de transaction est un **RETRAIT** et le solde de compte est inférieur au montant retiré + 2DT la méthode retourne : « On ne peut pas faire un retrait : Solde insuffisant»
- Si la condition d'ajout est vraie la méthode retourne un message respectant cette forme :
  « RETRAIT de 4.000 DT de compte 1 approuvé avec succès. »

`public String ajouterVersement(Transaction transaction);`(/1.5)

- Si le type de transaction est un **VERSEMENT** et le Destinataire n'est pas un compte **EPARGNE** le montant versé sera le montant versé – 2DT.
- Si la condition d'ajout est vraie la méthode retourne un message respectant cette forme :
  « Versement de 30.000 DT vers compte 2 approuvé avec succès. »

PS : Lorsque la transaction est approuvée le système va générer la date avant l'ajout et les frais des transactions doivent être pris en considération lors des modifications des soldes des comptes.

4) Nous souhaitons créer un service programmé automatiquement (scheduled) permettant d'afficher toutes les transactions d'aujourd'hui au niveau de console en utilisant la **journalisation** chaque **30 secondes** en respectant la signature suivante au niveau de **service (/2)** :

```
public void getAllTransactionByDate();
```

5) Afficher la liste des transactions d'une banque en utilisant le **JPQL** au niveau de **TransactionRepository** et en respectant la signature suivante au niveau de **service (/1.5)** :

```
public List<Transaction> getAllTransactionByBankId(long idBank);
```

6) Afficher la liste des transactions d'un type donné pour un compte donné en utilisant les **KEYWORD** au niveau de **TransactionRepository** et en respectant la signature suivante au niveau de **service (/1.5)** :

```
public List<Transaction>
getAllTransactionByTypeAndCompte(TypeTransaction type, long
idCompte);
```

PS : Le compte peut être un expéditeur ou un destinataire dans une transaction.

7) Créer un **Aspect** qui permet d'afficher la **date système** dans les logs pour chaque appel d'une méthode qui commence par « **get** » appartenant au package service. **(/1.5).**

Bon travail